Twitch Analytics API

Índice

Introducción **3**
Explicación del código **3**
Problemas y dificultades encontradas **4**
Problemas en el código 4
Problemas en el servidor 5
Implementación y ubicación del proyecto **5**

Introducción

Este proyecto consiste en consultar información de la API de twitch a través de la API que hemos creado nosotros. Se pueden realizar tres consultas distintas:

- **Consulta 1.** Solicitar la información de un usuario de twitch dado su ID.
  Para esta consulta simplemente ponemos en el buscador la siguiente url: http://35.180.210.32/analytics/user?id=39518378
  En esta url podemos modificar el id, para consultar la información de distintos usuarios.
- **Consulta 2**: Solicitar una lista de los directos que se están transmitiendo. Para ello, utilizaremos la siguiente url:
  http://35.180.210.32/analytics/streams
- **Consulta 3**: Solicitar una lista de los directos que se están transmitiendo ordenada de mayor a menor número de visualizaciones y con un límite de lista, es decir, si introducimos límite 3 se nos mostrarán los top 3 directos con más visualizaciones. Para ello utilizaremos la siguiente url: http://35.180.210.32/analytics/streams/enriched?limit=3
  En esta url podemos modificar el parámetro "limit" para seleccionar distintos tops; top 3, top 5, etc.

Hemos lanzado la API haciendo uso de amazon lightsail.

Explicación del código

Vamos a explicar cómo hemos realizado las llamadas a la API de twitch, y después las dificultades de cada caso.

Para realizar las llamadas a la API de twitch hemos utilizado la función curl().

Simplemente se inicializa el curl con curl_init() y después se fijan las funciones curl_setopt() con información sobre la url, los headers, etc.

Ejecutamos el curl con curl_exec() y obtenemos una respuesta. Para poder manejar los casos de error (400, 401, etc), utilizamos la función `curl_getinfo($ch, CURLINFO_HTTP_CODE)`.

Este código será necesario cada vez que realicemos una llamada a la API de twitch. Veamos ahora más detalladamente las distintas consultas:

- Consulta 1: Hemos creado una función getUsers() la cual recibe un token, un id de cliente y un id de streamer. Simplemente se realiza un curl a la API de twitch con la siguiente url:
  `$url = "https://api.twitch.tv/helix/users?id=$streamerId";`
- Consulta 2: Hemos creado una función getStreams() que recibe un token y un id de cliente. Se realiza un curl al igual que en la consulta anterior pero esta vez con la url correspondiente a la consulta de obtener lista de streams de twitch.
  En esta consulta teníamos que añadir algo más al código, ya que, no queríamos mostrar la información de todos los atributos de la consulta, solamente queríamos la información del título del stream, y el nombre de usuario del streamer. Para ello, lo que hemos realizado ha sido lo siguiente:
  - Decodificar la respuesta obtenida en el curl para poder leer y manejar los datos.

- Recorrer los datos y por cada dato guardamos en una variable nueva el título y el nombre de usuario.
- Finalmente codificamos la variable de respuesta creada y la mostramos.

❖ Consulta 3: Esta consulta es más compleja ya que tenemos que realizar dos llamadas a las API de twitch. Hemos creado tres funciones:

- **get_streams()**: Obtiene la lista de los streams en directo. No vamos a explicar esta función ya que se realiza de la misma forma que la consulta 2.
- **addUserInfo()**: Le pasamos una lista de streams que ya ha sido limitada, el token y el id de cliente. ç

  Empezamos creando una variable userIds que incluye los ids de usuarios que contiene la lista de streams. Como la API de twitch permite realizar peticiones de varios usuarios separándolos por un "&", creamos la url con los ids de la variable.

  Realizamos el curl y obtenemos la respuesta, esta respuesta al igual que en la consulta 2 tenemos que modificarla ya que no nos interesan todos los atributos.

  Ahora ya tenemos la variable $stream que contiene la lista de streams limitada y la variable $users que contiene la información de los usuarios.

  Finalmente creamos una variable $returnStream en la cual enlazamos la información de las dos variables que acabo de mencionar. Para ello, asignamos la información de la variable $stream a $returnStream y luego simplemente agregamos la información de $users. Eso se hace a través de un for each un poco complejo, por lo que hemos dejado comentarios en el código.
- **getEnrichedStreams()**: Esta función es la base la cual llama a las funciones que acabo de explicar. Simplemente llamamos primero a la función get_streams(), después la ordenamos por visualizaciones de mayor a menor, aplicamos el límite (el cual la función recibe), y finalmente llamamos a la función addUserInfo().

Para ir manejando los casos de error, nuestras funciones devuelven a parte de la respuesta, el código http que ha devuelto el curl.

En index.php simplemente generamos el token, obtenemos la url del servidor y llamamos a la función correspondiente (getUser, getStreams, getEnrichedStreams).

Problemas y dificultades encontradas

Problemas en el código

1) "SSL certificate problem": El primer error que tuvimos fue que el curl no encontraba un certificado ssl. Para solucionar esto agregamos las siguientes líneas de configuración del curl:

```
curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, false);
curl_setopt($ch, CURLOPT_SSL_VERIFYHOST, false);
```

Este problema también se podría solucionar creando una clave.

2) La primera vez que realizamos el código lo realizamos para que se ejecutara en la propia terminal de phpStorm, por lo que después, al crear el servidor, tuvimos que adaptar el código.

3) Otro error que tuvimos fue que habíamos generado el token de las credenciales de twitch en la terminal del ordenador, y luego lo que hacíamos era crear una variable con ese token. Esto era un problema, ya que, el token tiene fecha de expiración. Lo que hicimos fue crear una función getToken() en el index.php para obtener el token.

Problemas en el servidor

1) Creación de carpeta automática al clonar repositorio. Al clonar el repositorio del proyecto de git a nuestro servidor, se creaba una carpeta automática, por lo que, a la hora de acceder a la url, teníamos que añadir el nombre de la carpeta. La solución a este problema fue sencillo, simplemente movimos los archivos de la carpeta fuera de la carpeta, asegurándonos de que seguía funcionando el pull de git.
   Nuestro servidor coge la información de la carpeta "htdocs" por lo que movimos los archivos dentro de esa carpeta:

```
bitnami@ip-172-26-8-219:~/htdocs$ ls
caso1.php  caso2.php  caso3.php  documentacionAPI  index.php  README.md
```

2) El otro error que tuvimos con el servidor, fue que no conseguíamos que se detectara automáticamente nuestro archivo index.php, por lo que, cada vez que queríamos hacer una consulta teníamos que agregar /index.php a la url. Este problema nos costó bastante solucionarlo, ya que, nos cargamos el servidor intentando solucionarlo y no supimos arreglarlo. Al final tuvimos que crear un nuevo servidor.

   ¿Cómo lo solucionamos?
   Nos dimos cuenta de que al crear el servidor, se creaba también un archivo index.html el cual estaba escogido como por defecto. Nosotros lo habíamos borrado, ya que no queríamos tener ese archivo, pero en la configuración del servidor, seguía apareciendo ese archivo como archivo a escoger por defecto. Lo que hicimos fue cambiar ese archivo de configuración (httpd.conf).

```
#
<IfModule dir_module>
    DirectoryIndex index.php
</IfModule>
```

   Antes, el DirectoryIndex era index.html. Además de esto, hemos creado un archivo htaccess que redirige las solicitudes a index.php manteniendo los parámetros de la URL.

Implementación y ubicación del proyecto

Para probar esta implementación basta con escribir la url correspondiente a cada caso en el navegador:

Caso 1: http://35.180.210.32/analytics/user?id=39518378

Caso 2: http://35.180.210.32/analytics/streams

Caso 3: http://35.180.210.32/analytics/streams/enriched?limit=3

Al escribir la url se hace una petición GET a nuestra API, la cual posteriormente hará una petición a la API de twitch.

Para trabajar con nuestro proyecto, como se explica en el readme, simplemente hacemos un pull del repositorio de github a nuestros ordenadores y modificamos el código necesario. Después, lo probamos en local de la siguiente manera (ejemplo con la terminal de phpStorm):

```
PS C:\Users\Ainhoa\PhpstormProjects\twitch-analytics> php -S localhost:12000
[Fri Feb  7 15:21:53 2025] PHP 8.4.3 Development Server (http://localhost:12000) started
```

Después, simplemente accedemos a esa url y realizamos la petición que queramos como por ejemplo:

http://localhost:12000/analytics/streams/enriched?limit=3

Cuando nos aseguramos de que todo funcione, para hacer que funcione el servidor público haremos lo siguiente:

1) Realizar un commit a nuestro repositorio de github con los cambios.
2) Acceder a nuestra instancia en la página web de amazon lightsail.
3) Realizar un pull de github con los cambios actualizados.

El servidor almacena los códigos de index.php, caso1.php, caso2.php y caso3.php en la carpeta htdocs. Esta carpeta la podemos encontrar navegando por la terminal de nuestro servidor a la siguiente ruta: cd htdocs